

Analiza și Testarea Aplicației Tax Calculator

Proiect TSS 2026

Echipa de Proiect

5 mai 2026

Cuprins

- 1 Obiectivele Proiectului
- 2 Arhitectura Aplicației
- 3 Testare Funcțională
- 4 Acoperire Structurală
- 5 Mutation Testing
- 6 Utilizare GitHub Copilot
- 7 Demo și Validare
- 8 Concluzii

1. Obiectivele Proiectului

- Implementarea unui motor de calcul fiscal robust.
- Aplicarea tehnicilor de **Testare Funcțională**.
- Analiza **Acoperirii Structurale** (Statement & Branch).
- Evaluarea prin **Mutation Testing**.
- Integrarea tool-urilor **AI (GitHub Copilot)**.

2. Arhitectura Aplicației

- **Limbaj:** Python 3.13.2
- **Clasa principală:** TaxEngine
- **Funcția cheie:** `calculate_annual_tax`
- **Caracteristici:**
 - Validări riguroase (Income, Age, Category).
 - Dispatcher pe categorii (Salary, Business, Freelance, Crypto, etc.).
 - Sistem de reduceri compuse (Senior, Rezidență, Familie).

3. Testare Funcțională

- **Tehnici:** BVA (Boundary Value Analysis) & EP (Equivalence Partitioning).
- **Cazuri acoperite:**
 - Validări tipuri date.
 - Limite venit (0 - 1.000.000).
 - Praguri specifice pentru fiecare categorie de impozitare.
- **Rezultat:** Toate testele funcționale trecute cu succes.

4. Acoperire Structurală (CFG)

- **Complexitate Ciclomatică:** 41 (Grad F).
- **Graf de Control (CFG):** 31 de noduri principale.
- **Strategie:** Testarea drumurilor independente (Basis Path Testing).
- **Acoperire finală:**
 - **Statement:** 100%
 - **Branch:** 100%

5. Mutation Testing

- **Manual:** Generare de mutanți de ordin 1 (echivalenți, omorâți, supraviețuiți).
- **Automat:** Utilizarea mutmut.
- **Scor final: 79%.**
- **Îmbunătățiri:** Identificarea unor cazuri marginale neacoperite inițial de testele funcționale.

6. Utilizare GitHub Copilot

- **Rol:** Asistent în scrierea testelor unitare și calculul valorilor așteptate.
- **Beneficii:** Viteză crescută de implementare.
- **Atenție:** Necesită validare manuală a logicii matematice complexe.
- **Comparație:** AI-ul excelează la boiler-plate, dar omul identifică mai bine circuitele logice subtile.

7. Demo și Validare

- **Rulare teste:** `make test / make test-structural`
- **Generare rapoarte:** `make cov-statement-html`
- **Video Demo:** https://youtube.com/demo_placeholder

8. Concluzii

- Complexitatea ridicată necesită o strategie de testare multi-strat.
- Acoperirea 100% a instrucțiunilor nu este suficientă fără analiza ramurilor și a mutanților.
- Tool-urile AI sunt un suport valoros, dar nu înlocuiesc rigoarea inginerescă.

Mulțumim!

Întrebări?